

Clean Architecture & Cubit – Practical README

(بأمثلة بسيطة)

بنفس جديدة خطوة خطوة Feature إزاي بنشتغل الملف ده معمول عشان يبسط التفكير، ويشرح الكومنتات اللي بتكتبها، بس بشكل أوضح وأسهل.

(Mindset) فلسفة الشغل

قسم الشغل ساعتين ساعتين
افصل مهما كان وصلت لإيه

- مش كل حاجة ليها قواعد ثابتة.
- خليك بنفس كودك وطريقتك.
- التنظيم مش تعقيد... التنظيم راحة دماغ 🧠

When You Set Up New Feature

الفلو دايمًا كده (من غير لف):

```
Entity
├── Repository (Abstraction)
│   ├── UseCase
│   │   └── Cubit
│   │       └── UI
└── Repository (Implementation)
    ├── DataSource
    │   ├── Model (from json)
    │   └── Mapper
```

DOMAIN LAYER

Entity

Firebase أو API الداتا اللي التطبيق محتاجها. - من غير أي تفاصيل - يعني إيه؟

```
class UserEntity {
    final String id;
    final String name;

    UserEntity({required this.id, required this.name});
}
```

Repository (Abstraction)

الدومين مايعرفش الداتا جاية منين - ليه؟

```
abstract class AuthRepo {
    Future<UserEntity> login();
}
```

UseCase

بس Abstraction ينادي على - وظيفته

```
class LoginUseCase {
    final AuthRepo repo;

    LoginUseCase(this.repo);

    Future<UserEntity> call() {
        return repo.login();
    }
}
```

DATA LAYER

Model (from json)

مفيش لعب هنا 

```
class UserModel {
    final String? id;
    final String? name;
```

```

UserModel({this.id, this.name});

factory UserModel.fromJson(Map<String, dynamic> json) {
  return UserModel(
    id: json['id'],
    name: json['name'],
  );
}
}

```

Mapper (مهم جداً)

default values يحط - null يعالج - هو اللي

```

extension UserMapper on UserModel {
  UserEntity toEntity() {
    return UserEntity(
      id: id ?? '',
      name: name ?? 'Unknown',
    );
  }
}

```

Repository Implementation

```

class AuthRepoImpl implements AuthRepo {
  final AuthRemoteDataSource data;

  AuthRepoImpl(this.data);

  @override
  Future<UserEntity> login() async {
    final model = await data.login();
    return model.toEntity();
  }
}

```

Service Locator (مهم جداً)

الترتيب مهم

Register DataSource

```
Sl.registerLazySingleton<AuthRemoteDataSource>(  
    () => AuthRemoteDataSourceImpl(),  
);
```

Register Repo

```
Sl.registerLazySingleton<AuthRepo>(  
    () => AuthRepoImpl(Sl()),  
);
```

Dependency Inversion

Implementation مثنى Abstraction دايقاً استخدم

Cubit

BlocProvider

```
BlocProvider(  
    create: (_) => Sl<AuthCubit>(),  
    child: LoginPage(),  
)
```

👉 (أداء أحسن) value مثنى create استخدم

استخدام Cubit

```
final cubit = context.read<AuthCubit>();
```

Error Handling (ضروري)

```
Future<void> login() async {
  try {
    emit>Loading();
    final user = await loginUseCase();
    emit(Success(user));
  } catch (e) {
    emit(Error());
    return; // يكسر الدالة
  }
}
```

UI Layer

بس State يستقبل UI الـ

```
BlocBuilder<AuthCubit, AuthState>(
  builder: (context, state) {
    if (state is Loading) return CircularProgressIndicator();
    if (state is Success) return Text(state.user.name);
    return Container();
  },
)
```

Callback Functions (ببساطة)

```
void sayHello() {
  print('Hello');
}

void sayName(String name) {
  print('Hello $name');
}
```

```
onPressed: sayHello,
onPressed: () => sayName('Mohamed'),
```

مهمة Notes

- Cubit و Services في try / catch دائمًا
- State أي داتا جاية من برا تدخل
- في متغير state ممكن تحفظ:

```
final currentState = authCubit.state;
```



Golden Rule (Mapper)

- Model = زي ما الريموت باعت
- Entity = زي ما الدومين محتاج
- Mapper = هو اللي يصلح الدنيا



Reference

- Tharwat Samy – Fruits Hub